



## Scaleform 4.4 AS3 扩展参考

本文介绍 Scaleform 4.4

ActionScript 3.0

作者: Artem Bolgar, Prasad Silva

版本: 1.0

上次编辑: 2012 年 9 月 17 日

## 版权声明

### Autodesk® Scaleform® 4.4

© 2014 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and other countries: 123D, 3ds Max, Algor, Alias, AliasStudio, ATC, AutoCAD LT, AutoCAD, Autodesk, the Autodesk logo, Autodesk 123D, Autodesk CAM 360, Autodesk Homestyler, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSketch, AutoSnap, AutoTrack, Backburner, Backdraft, Beast, BIM 360, Burn, Buzzsaw, CADmep, CAiCE, CAMduct, CFdesign, Civil 3D, Cleaner, Combustion, Communication Specification, Configurator 360™, Constructware, Content Explorer, Creative Bridge, Dancing Baby (image), DesignCenter, DesignKids, DesignStudio, Discreet, DWF, DWG, DWG (design/logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DWGX, DXF, Ecotect, ESTmep, Evolver, FABmep, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, ForceEffect, FormIt, Freewheel, Fusion 360, Glue, Green Building Studio, Heidi, Homestyler, HumanIK, i-drop, ImageModeler, Incinerator, Inferno, InfraWorks, InfraWorks 360, Instructables, Instructables (stylized robot design/logo), Inventor, Inventor HSM, Inventor LT, Kynapse, Kynogon, LandXplorer, Lustre, MatchMover, Maya, Maya LT, Mechanical Desktop, MIMI, Mockup 360, Moldflow Plastics Advisers, Moldflow Plastics Insight, Moldflow, Moondust, MotionBuilder, Movimento, MPA (design/logo), MPA, MPI (design/logo), MPX (design/logo), MPX, Mudbox, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, Pixlr, Pixlr-o-matic, Productstream, Publisher 360, RasterDWG, RealDWG, ReCap, ReCap 360, Remote, Revit LT, Revit, RiverCAD, Robot, Scaleform, Showcase, Showcase 360, ShowMotion, Sim 360, SketchBook, Smoke, Socialcam, Softimage, Sparks, SteeringWheels, Stitcher, Stone, StormNET, TinkerBox, ToolClip, Topobase, Toxik, TrustedDWG, T-Splines, ViewCube, Visual LISP, Visual, VRED, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

### Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. "AS IS." AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

如何联系 Autodesk Scaleform:

---

|    |                                                                                        |
|----|----------------------------------------------------------------------------------------|
| 文档 | AS3 扩展参考                                                                               |
| 地址 | Autodesk Scaleform Corporation<br>6305 Ivy Lane, Suite 310<br>Greenbelt, MD 20770, USA |
| 网站 | <a href="http://www.scaleform.com">www.scaleform.com</a>                               |
| 电邮 | <a href="mailto:info@scaleform.com">info@scaleform.com</a>                             |
| 电话 | (301) 446-3200                                                                         |
| 传真 | (301) 446-3199                                                                         |

目录

1 引言 .....

# 1 引言

Autodesk Scaleform® SDK™ 是一种轻量级、高性能的富媒体 Adobe® Flash® 矢量图形引擎，它建立在专门针对游戏机和 PC 游戏开发者的洁净室实现基础之上。Autodesk Scaleform 将业经证明的可视化创作工具（如 Adobe Creative Suite®）与领先游戏开发者所需的最新硬件图形加速技术结合在一起。

由于 Flash 主要适用于 Web 而非游戏或应用程序开发，在某些方面（如 IME 输入的焦点处理、鼠标支持、文本字段支持和处理）具有有限的功能性。Autodesk Scaleform 通过将“扩展”（Extension）添加到 `ActionScript` 类来改进这些方面的基本功能。为了在 Scaleform 播放器中实现扩展支持，有必要将 `scaleform.gfx.Extensions.enabled` 属性设置为 `true`。

```
scaleform.gfx.Extensions.enabled = true;
```

或者

```
    导入 scaleform.gfx.*;  
    Extensions.enabled = true;
```

还有必要添加一个路径 `GFXSDK/Resources/AS3/CLIK` 作为 Flash 中的“源（Source）路径”，以使扩展类对 ActionScript 3.0 编译器可见（在 Flash Studio 中：‘Edit’（编辑）->‘Preferences’（首选项）->‘ActionScript’->‘ActionScript 3.0 Settings...’（ActionScript 3.0 设置...）->‘Source path’（Source 路径））。在多数情况下，开发者会希望将此语句添加到将用到扩展的 FLA 文件中的第一个帧。如果不设置此属性，Scaleform 播放器就会忽略对扩展的所有引用，试图实现最高水平的 Flash 兼容性。

开发者应明白，本文所述的扩展功能在标准 Flash 播放器中无效，因为仅在 Scaleform 中实现此扩展功能。与每个扩展关联的是一个 Scaleform 版本号，用以标识添加了相应扩展的播放器 SDK 的版本号。本文所述扩展在具有以前的版本号的 Scaleform 播放器中无法工作。

尽管 Autodesk 将竭尽全力使扩展 API 在所有不同版本中得到支持并保持一致，我们还是要保留在未来的 Scaleform 版本中更改、重命名或取消扩展 API 的权利。如果进行重大更改，我们会尽量在重点版本中进行，并会尽早发出通知。

## 2 扩展

### enabled

`enabled:Boolean [read-write]`

Scaleform 版本: 4.0.12

启用/禁用扩展。

### noInvisibleAdvance

`noInvisibleAdvance:Boolean [read-write]`

Scaleform 版本: 4.0.12

如果设置为 **true**，此属性会关闭所有看不见的电影剪辑的进展情况。这可能会用来提高包含许多隐藏的电影剪辑的 SWF 的性能。注意，Flash 推进看不见的电影剪辑（它仍然执行时间线动画，调用帧的 **ActionScript** 代码，等等）。因此，将此属性设置为 **true** 可能会导致 Scaleform 与 Flash 之间出现行为差异。

### getTopMostEntity()

```
public function getTopMostEntity(x :Number, y :Number,  
testAll:Boolean) :DisplayObject
```

Scaleform 版本: 4.0.13

返回一个可在 (x, y) 坐标（stage 坐标空间）中找到的最上面的 DisplayObject 实例。此方法在设置和未设置按钮处理程序（例如，CLICK、MOUSE\_DOWN、ROLL\_OVER 等）的字符之间可能有所不同。此区别对于筛选出没有处理程序来处理鼠标事件的字符来说可能会非常有用。

此方法与 `MovieClip.hitTest` 相反，因为 `hitTest` 检查 `x` 和 `y` 坐标是否在特定对象的内部，而且 `getTopMostEntity` 返回指定的 `x` 和 `y` 坐标处的实际对象。因此，`Extensions.getTopMostEntity(x, y).hitTest(x, y, true) == true`。

– 指明仅查找带有按钮处理程序的字符 (`false`)，或者查找任意字符 (`true`)。如果不指定此参数，则 `getTopMostEntity` 假设其为 `true`。

`x` : `Number` – 用来查找一个字符的 `Stage` 坐标。

### [getMouseTopMostEntity](#)

## **getMouseTopMostEntity()**

```
public function getTopMostEntity([testAll : Boolean],
    [mouseIndex : uint]) : DisplayObject
```

Scaleform 版本: 4.0.13

返回可在当前鼠标光标位置找到的一个最上面的 `DisplayObject` 实例。此函数类似于 `getTopMostEntity`，但它不使用显式坐标，而是使用鼠标坐标。

`testAll : Boolean` – 指明仅查找带有按钮处理程序的字符 (`false`)，或者查找任意字符 (`true`)。如果不指定此参数，则 `getTopMostEntity` 假设其为 `true`。

`Number` – 基于零的鼠标索引。

### [getTopMostEntity](#)

## **getMouseCursorType()**

```
public function getMouseCursorType([mouseIndex : uint]) : String
```

Scaleform 版本: 4.0.13

此静态方法返回用于指定鼠标控制器的当前鼠标光标类型。此方法类似于 `flash.ui.Mouse.cursor` 属性，唯一的区别是此方法允许对多个鼠标更改光标。

要跟踪并随意阻止鼠标光标变化，请使用 `MouseEvent.CURSOR_CHANGE` 扩展事件。

`mouseIndex :Number` – 基于零的鼠标索引（默认情况下为零）。

## 返回

返回光标的类型，参见 `flash.ui.MouseCursor` 静态常数，例如：

- `flash.ui.MouseCursor.ARROW : String = "arrow"`
- `flash.ui.MouseCursor.BUTTON : String = "button"`
- `flash.ui.MouseCursor.HAND : String = "hand"`
- `flash.ui.MouseCursor.IBEAM : String = "ibeam"`

[setMouseEventType](#)  
[MouseEvent](#)

## setMouseEventType() 静态方法

```
public function setMouseEventType(cursor : String, [mouseIndex : uint]) : void
```

Scaleform 版本： 4.0.13

此静态方法根据参数 `cursor` 更改鼠标光标。此方法类似于 `flash.ui.Mouse.cursor` 属性，唯一的区别是此方法允许对多个鼠标更改光标。

要跟踪并随意阻止鼠标光标变化，请使用 `MouseEvent.CURSOR_CHANGE` 扩展事件。

`cursor :uint` – 光标的类型，参见 `flash.ui.MouseCursor` 静态常数，例如：

- `flash.ui.MouseCursor.ARROW : String = "arrow"`
- `flash.ui.MouseCursor.BUTTON : String = "button"`
- `flash.ui.MouseCursor.HAND : String = "hand"`
- `flash.ui.MouseCursor.IBEAM : String = "ibeam"`

`mouseIndex :Number` – 基于零的鼠标索引（默认情况下为零）。



[getMouseCursorType](#)  
[MouseEvent](#)

## numControllers

numControllers:uint [read]

Scaleform 版本: 4.0.12

返回系统中检测到的控制器数量。

## setEdgeAAMode()

```
public function setEdgeAAMode (dispObj:DisplayObject, mode:uint):void
```

Scaleform 版本: 4.0.12

设置某个具体显示对象及其子对象的 EdgeAA 模式。接受下列模式值:

- scaleform.gfx.Extensions.EDGEAA\_INHERIT = 0; Inherit EdgeAA 模式 (来自父对象)。默认情况下为 On。
- scaleform.gfx.Extensions.EDGEAA\_ON = 1; Enable EdgeAA 模式, 用于目标显示对象及其子对象 – 除非禁用 (参见 ~~EDGEAA\_DISABLE~~ `EDGEAA_DISABLE`)
- scaleform.gfx.Extensions.EDGEAA\_OFF = 2; Do not use EdgeAA, 用于目标显示对象及其子对象。
- ~~scaleform.gfx.Extensions.EDGEAA\_DISABLE = 3; Disable EdgeAA, 用于目标显示对象及其子对象, 替代一个继承来的 EDGEAA\_ON 模式值。~~  
`scaleform.gfx.Extensions.EDGEAA_DISABLE = 3; Disable EdgeAA, 用于目标显示对象及其子对象, 替代一个继承来的 EDGEAA_ON 模式值。`

### 参数

dispObj:DisplayObject – 目标显示对象, 应用新的 EdgeAA 模式值。

mode:uint – EdgeAA 模式值。

另请参见:

[getEdgeAAMode](#)

## getEdgeAAMode()

```
static public function getEdgeAAMode (dispObj:DisplayObject):uint
```

Scaleform 版本: 4.0.12

从特定显示对象检索 EdgeAA 模式值。

`dispObj :DisplayObject` - 目标显示对象，检索 EdgeAA 模式值。

[setEdgeAAMode](#)

## 3 DisplayObjectEx 扩展

### setRendererString

```
static public function setRendererString(o:DisplayObject, s:String)
```

Scaleform 版本: 4.0.17

本属性允许在任何 MovieClip 实例中通过 ActionScript 发送自定义变量到渲染器。如果本属性置位，字符串值将作为用户数据传递给渲染器。

无默认值。

示例:

```
import scaleform.gfx.*;

// m is a MovieClip on stage.

DisplayObjectEx.setRendererString(m, "Abc");
trace(DisplayObjectEx.getRendererString(m));
```

### setRendererFloat

```
static public function setRendererFloat(o:DisplayObject, f:Number)
```

Scaleform 版本: 4.0.17

本属性允许在任何 MovieClip 实例中通过 ActionScript 发送自定义变量到渲染器。如果本属性置位，浮点值将作为用户数据传递给渲染器。

无默认值。

示例:

```
import scaleform.gfx.*;
```

```
// m is a MovieClip on stage.  
  
DisplayObjectEx.setRendererFloat(m, 17.1717);  
trace(DisplayObjectEx.getRendererFloat(m));
```

## **disableBatching**

```
static public function disableBatching(o:DisplayObject, b:Boolean)
```

Scaleform 版本: 4.0.17

此属性禁用针对自定义绘图的网格生成批处理。

### **示例**

```
import scaleform.gfx.*;  
  
// m is a MovieClip on stage.  
  
DisplayObjectEx.disableBatching(batchDisable, true);  
trace(DisplayObjectEx.isBatchingDisabled(batchDisable));
```

## 4 FocusEventEx 扩展

```
package scaleform.gfx
{
    import flash.events.FocusEvent;

    public final class FocusEventEx extends FocusEvent
    {
        public var controllerIdx : uint = 0;

        public function FocusEventEx(type:String) { super(type); }
    }
}
```

此事件是标准 `flash.events.FocusEvent` 的一个扩展。它添加一个 'controllerIdx' 成员，该成员指明导致该事件的控制器的一个基于零的索引。当把 `Extensions.enabled` 属性设置为 `true` 时，Scaleform 始终生成 `FocusEventEx` 事件，而不是标准 `FocusEvent`。用户可以检查收到的事件是不是 `FocusEventEx` 的一个实例，如果是，就把该事件对象归到扩展类型中。示例：

```
import scaleform.gfx.*;
import flash.events.FocusEvent;

Extensions.enabled = true;

function ev(e: FocusEvent)
{
    if (e is FocusEventEx)
    {
        var ee: FocusEventEx = e as FocusEventEx;
        trace("    controllerIdx = "+ee.controllerIdx);
    }
}

stage.addEventListener(FocusEvent.MOUSE_FOCUS_CHANGE, ev);
```

### controllerIdx

controllerIdx : uint [read]

Scaleform 版本： 4.0.12

指明哪个键盘/控制器用于该事件（基于零的索引）。

## 5 FocusManager 扩展

### alwaysEnableArrowKeys

```
alwaysEnableArrowKeys:Boolean [read-write]
```

Scaleform 版本: 4.0.12

`_focusrect` `false` 默认情况

下, 如果通过 `_focusrect = false` 禁用黄色焦点矩形, Flash 就不允许您使用箭头键更改焦点。要更改此行为, 请将 `alwaysEnableArrowKeys` 属性设置为 `true`。

### disableFocusKeys

```
disableFocusKeys:Boolean [read-write]
```

Scaleform 版本: 4.0.12

此静态属性禁用处理所有焦点键 (TAB、Shift-TAB 和箭头键), 因此, 用户可以实施自己的焦点键管理。

### moveFocus()

```
static public function moveFocus(keyToSimulate : String,  
                                startFromMovie:InteractiveObject = null,  
                                includeFocusEnabledChars : Boolean = false,  
                                controllerIdx : uint = 0) : InteractiveObject
```

Scaleform 版本: 4.0.12

此静态方法用来通过对以下焦点键之一进行模拟按键操作来移动焦点矩形: TAB、Shift-TAB 或箭头键。将此方法与 `disableFocusKeys` 和 `modalClip` 属性一起使用, 可实施自定义焦点管理。

`keyToSimulate :String` - 要模拟的键的名称: “向上”、“向下”、“向左”、“向右”、“tab”、“shifftab”。

`startFromMovie:InteractiveObject` - 可选参数, 用来指定一个字符; `moveFocus` 将它 (而非当前聚焦的参数) 用作一个起点。此属性可能是空 (Null) 或未定义, 这意味着当前聚焦的字符用作起点; 这可能会对指定第三个可选参数有用。

`includeFocusEnabledChars :Boolean` - 可选标志, 允许将 `moveFocus` 放在仅设置 `focusEnabled` 属性的字符以及设置了 `tabEnabled` / `tabIndex` 属性的字符上。如果不指定该标志, 或者将该标志设置为 `false`, 那么只有设置了 `tabEnabled` / `tabIndex` 属性的字符才能参加焦点移动。

`controllerIdx :uint` - 用于该操作的控制器的索引。如果不指定, 那就是用默认控制器 (控制器 0)。

返回下一个要聚焦的字符, 或者如果找不到该字符则返回 `null`。

[findFocus](#)  
[disableFocusKeys](#)  
[setModalClip](#)  
[getModalClip](#)

## findFocus()

```
static public function findFocus(keyToSimulate : String,  
                                parentMovie:DisplayObjectContainer = null,  
                                loop : Boolean = false,  
                                startFromMovie:InteractiveObject = null,  
                                includeFocusEnabledChars : Boolean = false,  
                                controllerIdx : uint = 0) : InteractiveObject
```

Scaleform 版本: 4.0.12

TAB、Shift-TAB 或箭头键

。将此方法与 `disableFocusKeys` 和 `setModalClip/getModalClip` 扩展相配合, 可用来实施自定义焦点管理。

`keyToSimulate :String` - 要模拟的键的名称: “向上”、“向下”、“向左”、“向右”、“tab”、“shifftab”。

`parentMovie:DisplayObjectContainer` - 电影剪辑，用作模式剪辑。焦点项搜索仅在此剪辑的子项内执行。可以为 `null`。

`loop :Boolean` - 要循环焦点的 `Boolean` 标志。例如，如果当前聚焦的项目位于底部，而键为“向下”键，那么 `findFocus` 要么返回“`null`”（如果此标志为“`false`”）或者返回最上面的可聚焦项目（如果此标志为“`true`”）。

`startFromMovie:InteractiveObject` - 可选参数，用来指定一个字符；`moveFocus` 将它（而非当前聚焦的参数）用作一个起点。此属性可能为 `null` 或未定义，这意味着当前聚焦的字符用作一个起点。

`includeFocusEnabledChars :Boolean` - 可选标志，允许将 `moveFocus` 放在仅设置 `focusEnabled` 属性的字符以及设置了 `tabEnabled` / `tabIndex` 属性的字符上。如果不指定该标志，或者将该标志设置为 `false`，那么只有设置了 `tabEnabled` / `tabIndex` 属性的字符才能参加焦点移动。As1!

`controllerIdx :uint` - 正



Scaleform 版本: 4.0.12

此静态方法返回与 `stage.focus` 属性相同的值。

`controllerIdx :uint` - 指明哪个键盘/控制器用于该事件（基于零的索引）。

当前聚焦的交互式对象。

## ~~numFocusGroups~~

`numFocusGroups() :uint` [read]

Scaleform 版本: 4.0.12

返回焦点组数量，通过调用 `setControllerFocusGroup` 函数进行设置。如果焦点组 0 和 3 处于活动状态，`numFocusGroups` 将返回 2。

## **setFocusGroupMask()**

`public function setFocusGroupMask(obj:InteractiveObject, mask:uint) :void`

Scaleform 版本: 4.0.12

此方法将一个位掩码设置为一个字符以及全部其子项。此位掩码将焦点组所有权指定给该字符，意思是只有该位掩码中表示的控制器才能将焦点移动到该字符内。使用 `setControllerFocusGroup` 扩展方法，可以将焦点组与控制器关联。

例如，我们假定“按钮 1”只能通过控制器 0 聚焦，而“电影剪辑 2”可通过控制器 0 和 1 聚焦。要实现此行为，请将焦点组与控制器进行关联：

```
FocusManager.setControllerFocusGroup(0, 0);  
FocusManager.setControllerFocusGroup(1, 1);
```

```
FocusManager.setFocusGroupMask(button1, 0x1); // 位 0 - 焦点组 0  
FocusManager.setFocusGroupMask(movieclip2, 0x1 | 0x2); // 位 0 和 1 - 焦点  
// 组 0 和组 1
```

“focusGroupMask” 位掩码可设置为母电影剪辑。这将会把此掩码值传播到所有其子项。

obj:InteractiveObject      一个交互式对象  
mask:uint                    - 一个焦点组位掩码

[setControllerFocusGroup](#)  
[getFocusGroupMask](#)

## getFocusGroupMask()

```
static public function getFocusGroupMask(obj:InteractiveObject) :uint
```

Scaleform 版本: 4.0.12

返回当前焦点组位掩码值（有关详细信息，请参见 setFocusGroupMask）。

obj:InteractiveObject      一个交互式对象

一个用于指定的交互式对象的焦点组位掩码。

[setControllerFocusGroup](#)  
[setFocusGroupMask](#)

## setControllerFocusGroup()

```
static public function setControllerFocusGroup(controllerIdx:uint,  
                                                focusGroupId:uint) : Boolean
```

Scaleform 版本: 4.0.12

此静态方法将 `controllerIndex` 表示的控制器与一个焦点组相关联。默认情况下，所有控制器均与焦点组 0 关联，这意味着它们正在使用同一焦点。不过，可以使每个控制器与其自己的焦点一起工作。例如，如果两个控制器应有单独的焦点（在分屏使用情况下），则 `setControllerFocusGroup(1,1)` 将为其控制器 1 创建一个单独的焦点组。调用 `setControllerFocusGroup(1,0)` 将使控制器 0 和 1 再次共享同一焦点。

`controllerIdx:uint` - 控制器的零基索引。  
`focusGroupIdx:uint` - 焦点组的零基索引。

如果成功，就返回 `true`。

## **getControllerFocusGroup()**

```
static public function getControllerFocusGroup(controllerIdx:uint) :uint
```

Scaleform 版本： 4.0.12

此方法返回与指定控制器关联的焦点组索引。

`controllerIndex` - 物理控制器的零基索引。

焦点组的基于零的索引。

## **setModalClip()**

```
static public function setModalClip(mc:Sprite, controllerIdx:uint = 0) : void
```

Scaleform 版本： 4.0.12

此静态方法将指定的电影剪辑设置为用于焦点管理的“模式” (modal) 剪辑。这意味着 TAB、Shift-TAB 以及箭头键将仅在所有“table”子项之间的指定电影剪辑内移动焦点。

`controllerIdx:uint` - 控制器的一个零基索引。  
`mc:Sprite` - 一个模式剪辑。

## **getModalClip()**

```
static public function getModalClip(controllerIdx:uint = 0) : Sprite
```

Scaleform 版本: 4.0.12

此静态方法返回用于指定控制器的模式剪辑。

`controllerIdx` - 控制器的零基索引。

一个模式剪辑，如果未找到，则表明未定义。

## **6 GamePad 扩展**

### **控件常数**

此类为一般游戏手柄控件（如触发器、模拟摇杆和按钮）提供辅助常数。他们是 `SF_KeyCodes.h` 中定义的常数的镜面反射。Scaleform 在运行时插入正确的值，因此，编译利用这些常数的 SWF 的操作就会按预期进行。

为了方便起见，Scaleform FxPlayer 框架将游戏手柄控件映射到键盘当量，然而，自定义的集成或应用程序可将这些常数与 Gfx::Movie::HandleEvent 一起使用，如果对于 AS3 键盘事件有必要的话，在控制器与键盘之间产生某种区别。

Scaleform FxPlayer 框架确实将这些常数与 GamePadAnalogEvents 一起使用，为这样的模拟值提供适当反馈。不过，也可以想到的是，开发者也可以不使用 GamePad 常数，而是使用键盘代码。是否选择将游戏手柄事件与键盘事件结合在一起，留给开发者自行做出判断。

## **supportsAnalogEvents()**

```
public static function supportsAnalogEvents() :Boolean
```

Scaleform 版本： 4.0.13

假如基本硬件平台的 Scaleform 实现支持游戏手柄模拟事件（如对于触发器和拇指棒），就返回 true。此值可用来确定是否支持 GamePadAnalogEvents。

[GamePadAnalogEvent](#)

## 7 GamePadAnalogEvent 扩展

```
package scaleform.gfx
{
    import flash.events.Event;

    public final class GamePadAnalogEvent extends Event
    {
        public static const CHANGE:String = "gamePadAnalogChange";

        public var code :uint = 0;    // 参见 scaleform.gfx.GamePad, 了解
// 有效手柄代码
        public var controllerIdx : uint = 0;
        public var xvalue :Number = 0;    // 标准化 [-1, 1]
        public var yvalue :Number = 0;    // 标准化 [-1, 1]

        public function GamePadAnalogEvent(bubbles:Boolean, cancelable:Boolean,
                                           code:uint, controllerIdx:uint = 0,
                                           xvalue:Number = 0, yvalue:Number = 0)
        {
            super(GamePadAnalogEvent.CHANGE, bubbles, cancelable);
            this.code = code;
            this.controllerIdx = controllerIdx;
            this.xvalue = xvalue;
            this.yvalue = yvalue;
        }
    }
}
```

如果 Scaleform 支持针对某个特定平台的游戏手柄模拟事件，就触发此事件。此事件只从 Stage 发出，而所有侦听者都会附加到 Stage 对象。Scaleform FxPlayer 框架通过用于 'code' 属性的适当 GamePad 常数触发这些事件，不过，如有必要，开发者可以在自己的 Scaleform 集成中使用其它值（如键盘值）。

```
import scaleform.gfx.*;

trace("GamePadAnalogEvents supported? " + GamePad.supportsAnalogEvents());

function ev(e: GamePadAnalogEvent)
{
    trace("code = " + ev.code);
    trace("controllerIdx = " + ev.controllerIdx);
    trace("xvalue = " + ev.xvalue);
    trace("yvalue = " + ev.yvalue);
}
```

```
}  
stage.addEventListener(GamePadAnalogEvent.CHANGE, ev);
```

## **code**

code : uint

Scaleform 版本: 4.0.13

指明使用哪个键/控件生成此事件。Scaleform FxPlayer 框架将使用游戏手柄常数。参见 [GamePad](#)。

## **controllerIdx**

controllerIdx : uint

Scaleform 版本: 4.0.13

指明哪个键盘/控制器用于该事件（基于零的索引）。

## **xvalue**

xvalue : Number

Scaleform 版本: 4.0.13

指明 x 轴中的当前值。该值将在 -1 与 1 之间标准化，其中包含 -1 和 1。

## **yvalue**

yvalue : Number

Scaleform 版本: 4.0.13

指明 y 轴中的当前值。该值将在 -1 与 1 之间标准化，其中包含 -1 和 1。

## 8 InteractiveObjectEx 扩展

### getHitTestDisable()

```
public function getHitTestDisable(o:InteractiveObject) : Boolean
```

Scaleform 版本: 4.0.13

返回 'hitTestDisable' 标志的状态。当设置为 true 时, MovieClip.hitTest 函数在命中测试检测中忽略此交互式对象。此外, 所有其它鼠标事件都传播到该对象。

默认值为 false。

o - An interactive object.

一个代表 'hitTestDisable' 标志的状态的 Boolean 值。

[InteractiveObjectEx.setHitTestDisable](#)

### setHitTestDisable()

```
public function setHitTestDisable(o:InteractiveObject, f:Boolean) : void
```

Scaleform 版本: 4.0.13

设置 'hitTestDisable' 标志的状态。当它设置为 true 时, MovieClip.hitTest 函数在命中测试检测中忽略此交互式对象。

默认值为 false。

o - 一个交互式对象。

f - 一个代表 'hitTestDisable' 标志的新状态的 Boolean 值。



[InteractiveObjectEx.getHitTestDisable](#)

## **getTopmostLevel()**

```
public function getTopmostLevel (o:InteractiveObject) : Boolean
```

Scaleform 版本: 4.0.13

返回 'topmostLevel' 标志的状态。设置为 `true` 时, 此字符显示在所有其它字符的上面, 而不管其深度如何。

o - An interactive object.

一个代表 'topmostLevel' 标志的状态的 Boolean 值。

[InteractiveObjectEx.setTopmostLevel](#)

## **setTopmostLevel ()**

```
public function setTopmostLevel(o:InteractiveObject, f:Boolean) : void
```

Scaleform 版本: 4.0.13

设置 'topmostLevel' 标志的状态。设置为 `true` 时, 此字符显示在所有其它字符的上面, 而不管其深度如何。当把光标从各个级别拉到对象上面时, 这可能对于实现自定义鼠标光标有用。默认值为 `false`。

如果把多个字符标为 "topmostLevel", 则拖曳顺序与没有将字符标为最上面时相同, 也就是说, 如果对象 A 被拖到对象 B 下面, 则在把上述字符标为最上面时, 对象 A 仍将会在对象 B 下面, 而不管把 "topmostLevel" 属性设置为 `true` 的顺序如何。

注意: 一旦某个字符标为 "topmostLevel", `swapDepth` ActionScript 函数就不会对此字符产生任何影响。默认值为 `false`。

一个交互式对象。

f - 一个代表 'topmostLevel' 标志的新状态的 Boolean 值。

[InteractiveObjectEx.getTopmostLevel](#)

## 9 KeyboardEventEx 扩展

```
package scaleform.gfx
{
    import flash.events.KeyboardEvent;

    public final class KeyboardEventEx extends KeyboardEvent
    {
        public var controllerIdx : uint = 0;

        public function KeyboardEventEx(type:String) { super(type); }
    }
}
```

此事件是标准 `flash.events.KeyboardEvent` 的一个扩展。它添加一个 'controllerIdx' 成员，该成员指明导致该事件的控制器的一个基于零的索引。当把 `Extensions.enabled` 属性设置为 `true` 时，Scaleform 始终生成 `KeyboardEventEx` 事件，而不是标准 `KeyboardEvent`。用户可以检查收到的事件是不是 `KeyboardEventEx` 的一个实例，如果是，就把该事件对象归到扩展类型中。

```
import scaleform.gfx.*;
import flash.events.KeyboardEvent;

Extensions.enabled = true;

function ev(e: KeyboardEvent)
{
    if (e is KeyboardEventEx)
    {
        var ee: KeyboardEventEx = e as KeyboardEventEx;
        trace("    controllerIdx = "+ee.controllerIdx);
    }
}

stage.addEventListener(KeyboardEvent.KEY_DOWN, ev);
stage.addEventListener(KeyboardEvent.KEY_UP, ev);
```

### controllerIdx

`controllerIdx : uint [read]`

Scaleform 版本: 4.0.12

指明哪个键盘/控制器用于该事件（基于零的索引）。

## 10 MouseCursorEvent 扩展

```
package scaleform.gfx
{
    import flash.events.Event;

    public final class MouseCursorEvent extends Event
    {
        public var cursor : String = "auto";
        public var mouseIdx : uint = 0;

        static public const CURSOR_CHANGE : String = "mouseCursorChange";

        public function MouseCursorEvent()
        {
            super("MouseCursorEvent", false, true);
        }
    }
}
```

此事件用来跟踪和/或防止鼠标光标变化。它设置有以下事件的属性：

bubbles – false，它不起泡 (bubble)

cancellable – true，可通过调用 preventDefault() 方法来防止默认操作（光标变化）。

此事件对于实施自定义的动画鼠标光标非常有用。只要 Scaleform 更改鼠标光标的形状（通过翻转文本字段或按钮，或者通过设置 flash.ui.Mouse.cursor 属性），都会对一个 **stage** 触发此事件（假如将 Extensions.enabled 设置为 true 的话）。

```
Extensions.enabled = true;

function e(e:MouseCursorEvent)
{
    trace(e.type + " " + e.mouseIdx + " " + e.cursor);
    e.preventDefault();
}
stage.addEventListener(MouseCursorEvent.CURSOR_CHANGE, e);
```

只有在一个 Stage 上设置了侦听器，才触发此事件。

## **cursor**

`cursor : String [read]`

Scaleform 版本: 4.0.13

此属性指明要更改为的光标的类型。它包含来自 `flash.ui.MouseCursor` 类的字符串值之一，例如：

- `flash.ui.MouseCursor.ARROW : String = "arrow"`
- `flash.ui.MouseCursor.BUTTON : String = "button"`
- `flash.ui.MouseCursor.HAND : String = "hand"`
- `flash.ui.MouseCursor.IBEAM : String = "ibeam"`

## **mouseIdx**

`mouseIdx : uint [read]`

Scaleform 版本: 4.0.13

指明为哪个鼠标/控制器生成该事件（基于零的索引）。

## 11 MouseEventEx 扩展

```
package scaleform.gfx
{
    import flash.events.MouseEvent;

    public final class MouseEventEx extends MouseEvent
    {
        public var mouseIdx : uint = 0;
        public var nestingIdx : uint = 0;
        public var buttonIdx : uint = 0; // LEFT_BUTTON, RIGHT_BUTTON, ...

        public static const LEFT_BUTTON : uint = 0;
        public static const RIGHT_BUTTON : uint = 1;
        public static const MIDDLE_BUTTON : uint = 2;

        public function MouseEventEx(type:String) { super(type); }
    }
}
```

此事件是标准 `flash.events.MouseEvent` 的一个扩展。它为支持多控制器和鼠标右/中键添加属性。当把 `Extensions.enabled` 属性设置为 `true` 时，`Scaleform` 始终生成 `MouseEventEx` 事件，而不是标准 `MouseEvent`。用户可以检查收到的事件是不是 `MouseEventEx` 的一个实例，如果是，就把该事件对象归到扩展类型中。

```
import scaleform.gfx.*;
Extensions.enabled = true;

stage.doubleClickEnabled = true;

function ev(e:MouseEvent):void
{
    trace("!!!! EVENT. " + cnt++);
    trace("    eventType      = "+e.type);
    trace("    bubbles        = "+e.bubbles);
    trace("    eventPhase      = "+e.eventPhase);
    trace("    target          = "+e.target.name);
    trace("    currentTarget   = "+e.currentTarget.name);
    if (e is MouseEventEx)
    {
        var ee:MouseEventEx = e as MouseEventEx;
        trace("    mouseIdx        = "+ee.mouseIdx);
        trace("    nestingIdx      = "+ee.nestingIdx);
    }
}
```

```

        trace("    buttonIdx    = "+ee.buttonIdx);
    }
    trace(e);
}
stage.addEventListener(MouseEvent.MOUSE_DOWN, ev);
stage.addEventListener(MouseEvent.MOUSE_UP, ev);
stage.addEventListener(MouseEvent.CLICK, ev);
stage.addEventListener(MouseEvent.DOUBLE_CLICK, ev);

```

一旦启用扩展，就会为鼠标右、中、中心等键触发鼠标事件，而且用户必须检查 `buttonIdx` 属性，搞清楚哪个按钮生成该事件。

## buttonIdx

`buttonIdx : uint [read]`

Scaleform 版本： 4.0.13

指明为哪个按钮生成该事件（基于零的索引）。此值可以为下列值之一：

- `MouseEventEx.LEFT_BUTTON :uint = 0` - 鼠标左键
- `MouseEventEx.RIGHT_BUTTON :` - 鼠标右键
- `MouseEventEx.MIDDLE_BUTTON :` - 鼠标中键
- 假如鼠标有 3 个按键，大于 2 的任何值都是合法的。

## mouseIdx

`mouseIdx : uint [read]`

Scaleform 版本： 4.0.13

指明为哪个鼠标/控制器生成该事件（基于零的索引）。

## nestingIdx

`nestingIdx : uint [read]`

Scaleform 版本: 4.0.13

此属性对于 rollOver/Out、mouseOver/Out 和 dragOver/Out 事件来说是可选的。此参数指定同一字符上的嵌套的翻转/拖出 (rollover/dragover) 事件的索引。

当生成嵌套的 rollOver/rollOut、mouseOver/Out 和 dragOver/dragOut 事件（单独针对每个鼠标光标），此参数代表嵌套的基于零的索引：初始事件将会把 0 作为参数；如果第二个光标翻转同一个字符，则把该成员设置为 1，就会触发第二个 rollOver/rollOut 事件。如果有任何光标离开该字符，该成员设置为 1，就会触发 rollOut/mouseOut/dragOut；把该成员设置为 0，就会触发最后一个 rollOut/mouseOut/dragOut。

## 12 TextEventEx 扩展

```
package scaleform.gfx
{
    import flash.events.TextEvent;

    public final class TextEventEx extends TextEvent
    {
        public var controllerIdx : uint = 0;

        public function TextEventEx(type:String) { super(type); }
    }
}
```

此事件是标准 `flash.events.TextEvent` 的一个扩展。它添加一个 `'controllerIdx'` 成员，该成员表明导致该事件的控制器的一个基于零的索引。当把 `Extensions.enabled` 属性设置为 `true` 时，Scaleform 始终生成 `TextEventEx` 事件，而不是标准 `TextEvent`。用户可以检查收到的事件是不是 `TextEventEx` 的一个实例，如果是，就把该事件对象归到扩展类型中。

```
import scaleform.gfx.*;
import flash.events.TextEvent;

Extensions.enabled = true;

function ev(e: TextEvent)
{
    if (e is TextEventEx)
    {
        var ee: TextEventEx = e as TextEventEx;
        trace("    controllerIdx = "+ee.controllerIdx);
    }
}

txf.addEventListener(TextEvent.TEXT_INPUT, ev);
```

### controllerIdx

`controllerIdx : uint [read]`

Scaleform 版本: 4.0.12



指明哪个键盘/控制器用于该事件（基于零的索引）。

## **LINK\_MOUSE\_OVER/LINK\_MOUSE\_OUT**

```
public static const LINK_MOUSE_OVER:String = "linkMouseOver";  
public static const LINK_MOUSE_OUT:String = "linkMouse";
```

Scaleform 版本： 4.0.14

开发者现在能够使用 TextFieldEx.LINK\_MOUSE\_OVER 和 TextFieldEx.LINK\_MOUSE\_OUT 事件扩展侦听 TextField 链接（由 HTML 标记指定）上的鼠标 over/out 事件。这些事件的行为与其它 AS3 事件相同，而且可以使用 addEventListener 范例进行侦听。

## 13 System 扩展

### **actionVerbose**

`actionVerbose:Boolean` [read-write]

Scaleform 版本: 4.0.12

/ 操作码跟踪。

### **getStackTrace()**

`public function getStackTrace() : String`

Scaleform 版本: 4.0.12

获取格式化为字符串的当前堆栈跟踪。

### **getCodeFileName()**

`public function getCodeFileName() : String`

Scaleform 版本: 4.0.12

获取当前执行的代码的文件名。

## 14 TextFieldEx 扩展

### appendHtml()

```
public function appendHtml(textField:TextField, newHtml:String) :void
```

Scaleform 版本: 4.0.12

将 newHtml 参数指定的 HTML 附加到文本字段的文本末尾。此方法比一个 htmlText 属性上的添加指定 (+) (如 txt.htmlText += moreHtml) 更加有效。htmlText 属性上的常规 += 生成 HTML 字符串, 附加新的 HTML 部分, 然后从头解析整个 HTML。此函数执行增量 HTML 解析, 即它仅解析来自 newHtml 字符串参数的 HTML (这就是 newHtml 参数中的 HTML 有理由的原因, 这对于 += 操作符并非必需的)。它对于包含大量内容的文本字段尤其重要。

如果将一个样式表应用到该文本字段, 此方法就无效。

textField:TextField - 将 HTML 附加到的一个文本字段。  
newHtml:String - 要将 HTML 附加到现有文本的字符串。

### setIMEEnabled()

```
static public function setIMEEnabled(textField:TextField, isEnabled:Boolean): void
```

Scaleform 版本: 4.0.12

启用/禁用对指定文本字段的 IME。如果将 isEnabled 设置为 false, 则不让在此文本字段中激活 IME。默认情况下, 启用了 IME。

textField :TextField - 要操作的文本字段。  
isEnabled :Boolean - 如果是 true - 启用 IME; 否则禁用。

## setVerticalAlign()

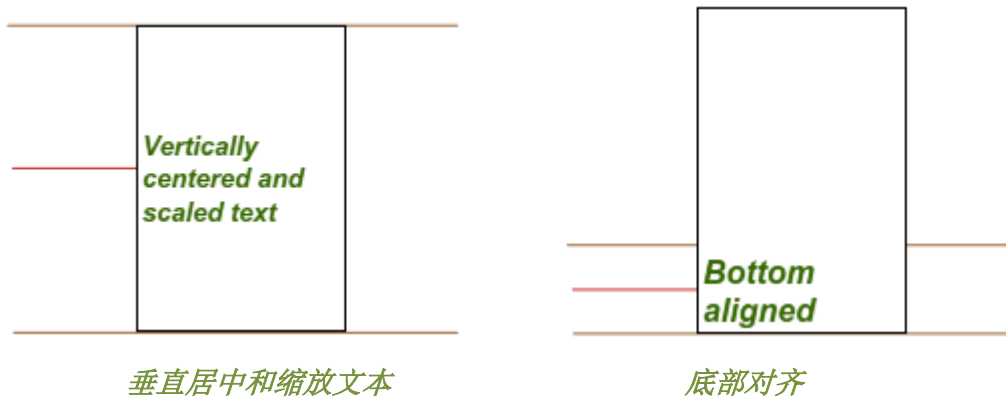
```
public function setVerticalAlign(textField:TextField, valign:String) :void
```

Scaleform 版本: 4.0.12

在文本框内设置文本垂直对齐。该属性的有效值为 TextFieldEx 类中声明的下列常数:

```
public static const VALIGN_TOP:String      = "top";  
public static const VALIGN_CENTER:String  = "center";  
public static const VALIGN_BOTTOM:String  = "bottom";
```

如果将此属性设置为 center, 则文本位于文本框内中心; 如果设置为 bottom, 则文本位于文本框内底部 (参见下图):



默认值为 top

textField:TextField - 设置垂直对齐的文本字段

valign:String - 对齐值 ("top", "center", "bottom")。